Uzyra book reader — how pages are shown, loaded and turned today

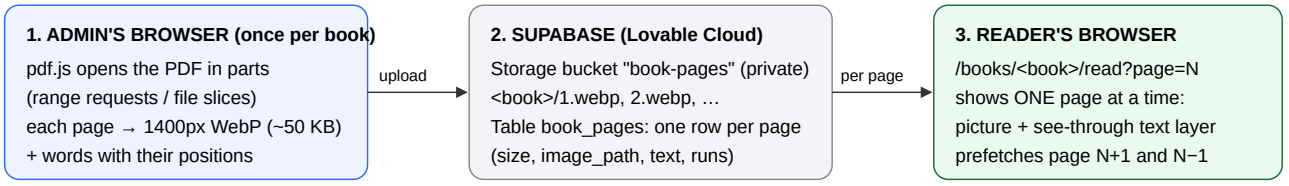
A document for researching a better design. Website: uzyra / ritajet.com, a language-learning site (Arabic UI, right-to-left) where learners read real English books and long PDFs (novels, lecture slides) up to 500 MB, inside the site, next to an AI tutor ("Rita").

بالعربي باختصار: القارئ حالياً بيعرض صفحة وحدة كل مرة (صورة خفيفة للصفحة وفوقها نص شفاف للتحديد). لما تفتح صفحة بنحمّل الصفحة اللي بعدها واللي قبلها بالخلفية. التقليب بأزرار «السابقة/التالية» أو كتابة رقم الصفحة أو الأسهم بالكيبورد. السؤال: هل هاد أحسن حل؟ كم صفحة لازم نعرض ونحمّل؟ وكيف نخلي التقليب سريع وممتع وما يملّ المستخدم، خصوصاً على الآيباد والتلفون؟

1. The goal and the hard rules (constraints any solution must respect)

- **Never download the whole PDF** to the learner (books are up to 500 MB, read on iPads and phones). The site must never freeze.
- The learner must be able to **select real text** (a word or a sentence) on the page, then press a floating circle to ask the tutor: explain, meaning and pronunciation, break the sentence down, flashcards, multiple-choice questions. The server checks the selection against the page's stored text.
- Layout like the study page: tutor column on the left, the book on the right (a draggable divider on tablets/desktop; two tabs on phones).
- Backend: Supabase (hosted by Lovable Cloud). Files are **private**; the browser gets **signed URLs** with the reader's own session. No new serverless (Edge) functions, **no background/cron jobs**, no server-side rendering service (cost: the database must be able to sleep when nobody visits).
- Front end: React 19 + TanStack Start/Router + React Query, Tailwind, pdf.js (pdfjs-dist) only in the admin's browser. Main devices: iPad Safari and phones; Chrome on desktop.
- Arabic interface (RTL), books mostly English left-to-right (some Arabic).

2. How it works today (architecture)



What ONE page costs the reader today (3 requests, one after another):

① GET book_pages row (text + runs) → ② POST sign → a 1-hour signed URL → ③ GET the WebP picture (~50 KB)

Measured with 250 ms per request: next page (already prefetched) 70–240 ms · jump to page 40 (not prefetched) ≈ 830 ms
5 quick taps on «next» (pages 3→8): the last page appeared 1.4 s after the first tap (only ±1 page is prefetched).

2.1 Preparing a book (admin, once)

The admin presses «جهّز الكتاب للقراءة» ("prepare the book for reading"). Their browser opens the PDF with pdf.js *in parts* (HTTP range requests from a signed link, or slices of the local file), and for each page: renders it to a canvas

1400 px wide, saves it as WebP (JPEG on Safari, which cannot encode WebP), reads the page's words with their positions (`getTextContent`), uploads the picture to the private bucket `book-pages/<book id>/<page>.webp` and writes a row to the table `book_pages` . Two uploads run while the next page is drawn. It can be stopped and continued. Measured: 80 photo slides in ~10 s on a desktop, pictures 8–86 KB (average ~60 KB). When all pages are in, `books.prepared_at` is set and readers can open the book.

book_pages column	meaning
book_id, page	primary key
width, height	picture size in pixels (e.g. 1400×1812; slides 1400×788)
image_path	path in the private bucket
text	the page's text in reading order (up to 60,000 chars)
runs	JSON list of text pieces: [text, left, top, width, height] as fractions of the page (0–1); used to lay the see-through text over the picture
has_text	false for scanned pages (picture only, nothing selectable; no OCR yet)



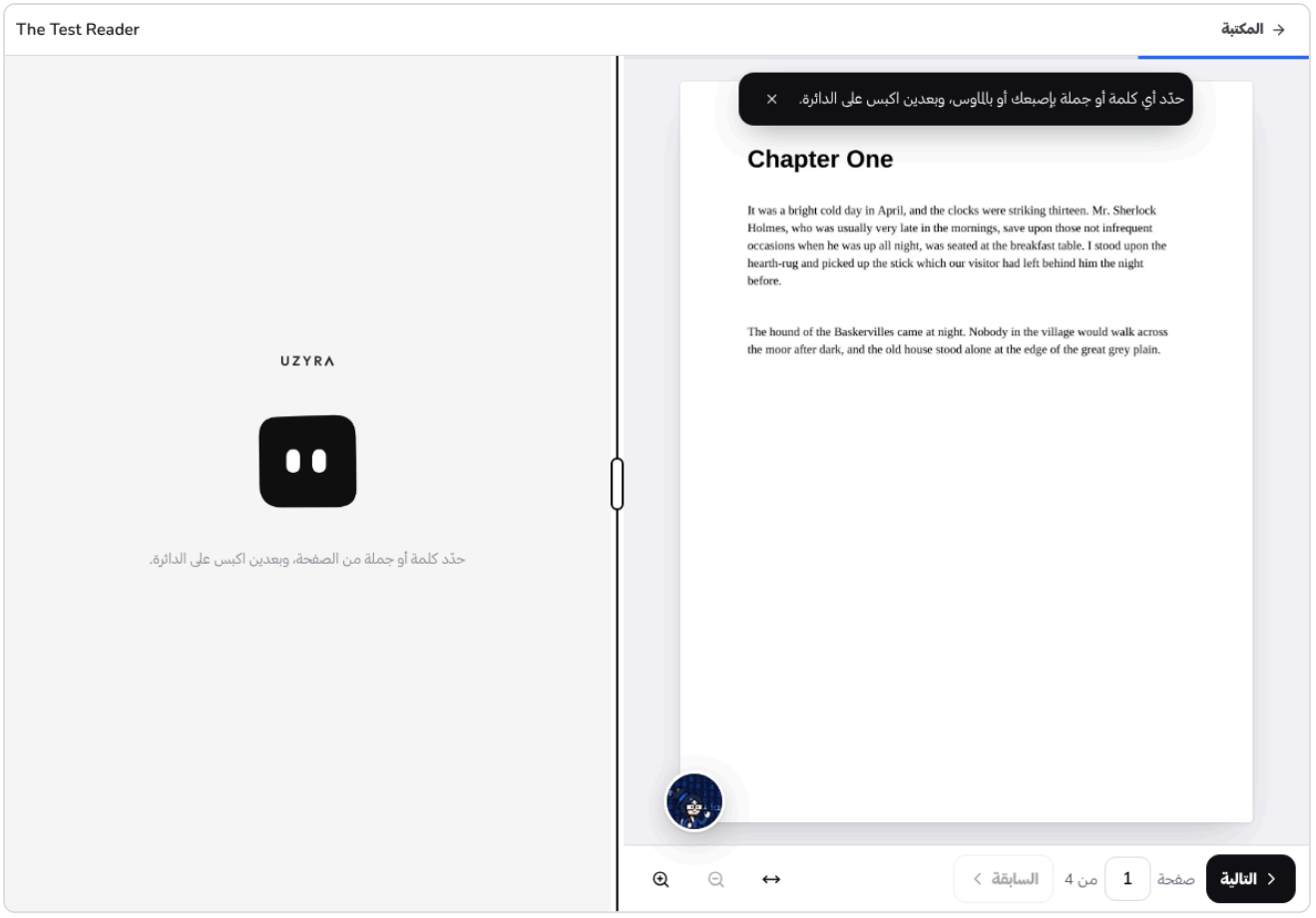
Admin: the «القراءة داخل الموقع» card after preparing a book (here 80 slides).



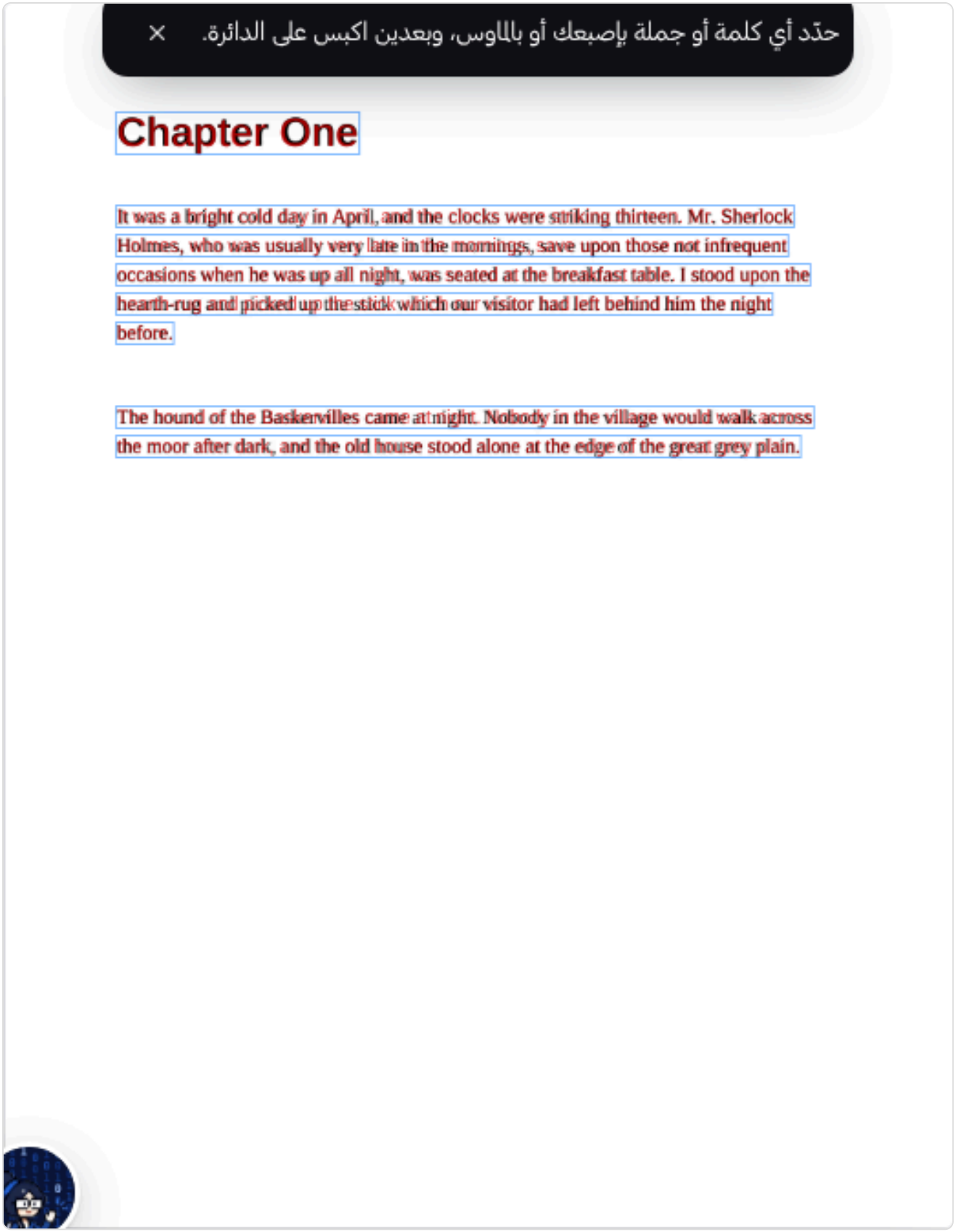
Admin: books still waiting to be prepared.

2.2 Reading: what is shown

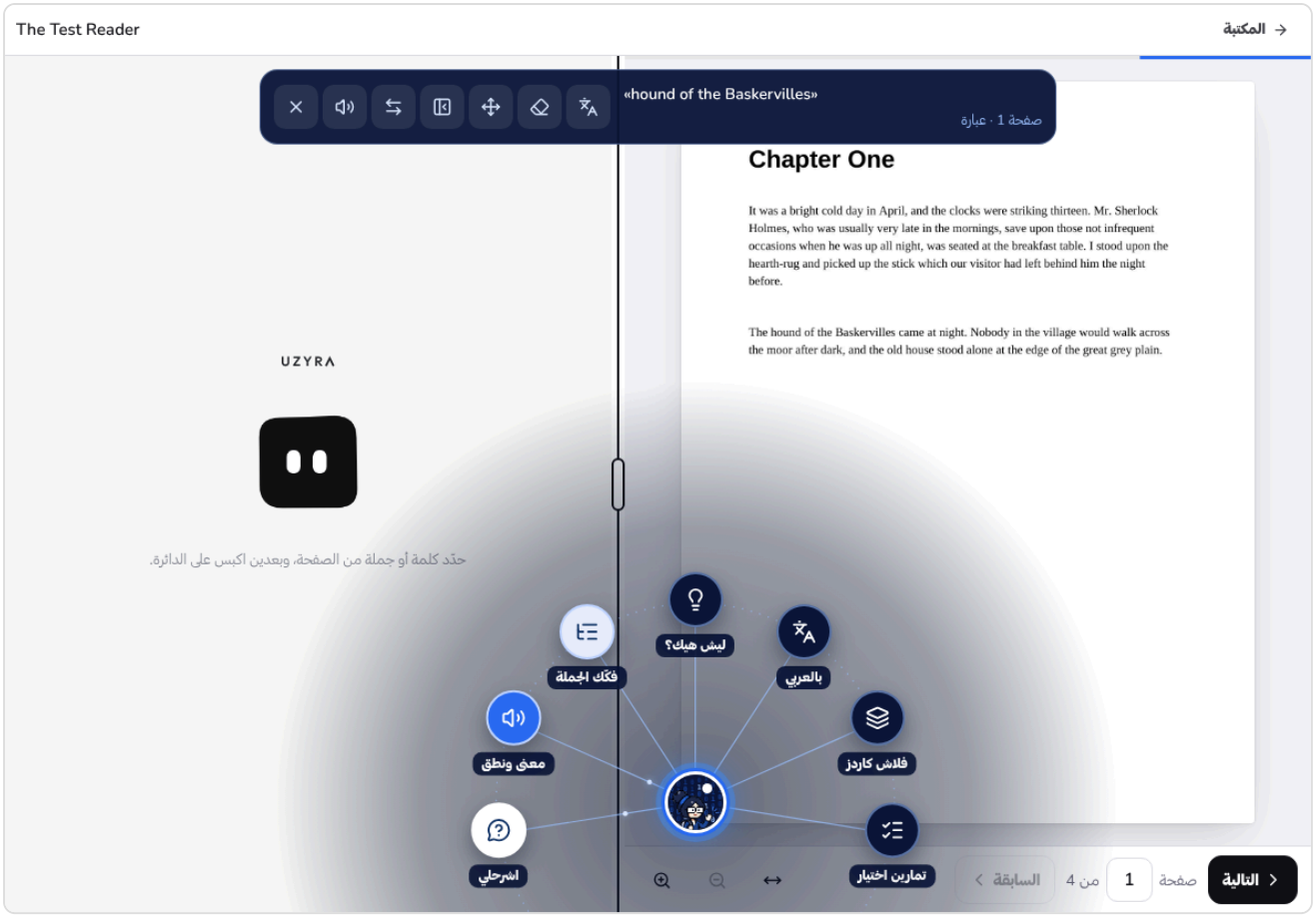
- **One page at a time** (paged mode, not a scrolling list of pages). By default the whole page fits in view; a button switches to "fit width"; 5 zoom steps (1×–2.5×); a zoomed page scrolls inside its pane.
- The page is an `<img>` of the WebP picture. Over it, a transparent text layer: one absolutely-positioned `<span>` per text piece, font size in container units, stretched horizontally (`scaleX`) to cover its words exactly (the same idea as pdf.js' textLayer). This makes native selection work: double-click a word, drag across lines (a blank layer under the words stops the selection jumping across gaps).
- A 3 px progress line at the top (page / total). A bar under the page: «السابقة» (previous) · "page [n] of N" (editable) · «التالية» (next), plus fit/zoom buttons.



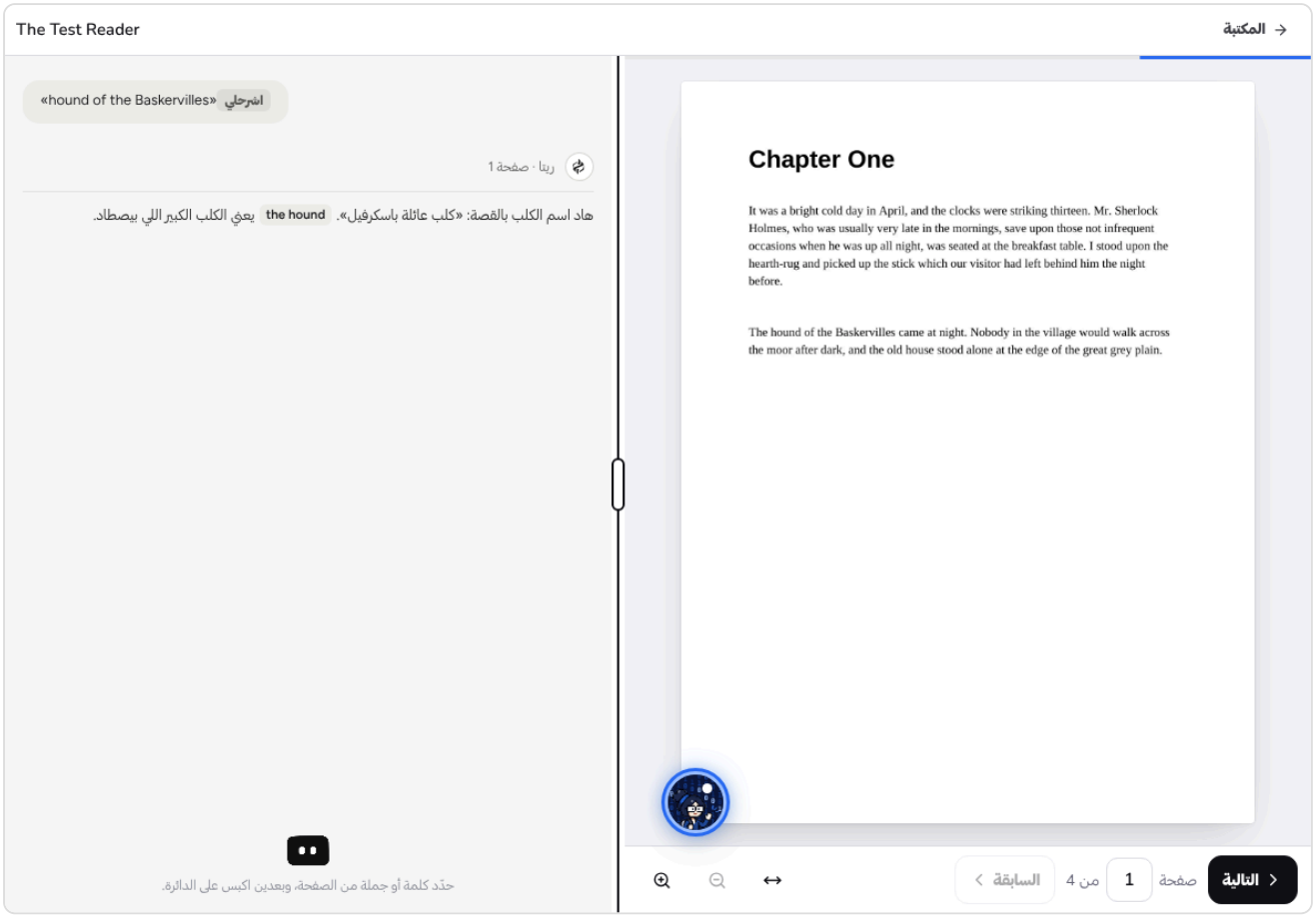
iPad (1180×820): tutor left, page right, whole page in view; first-visit tip at the top.



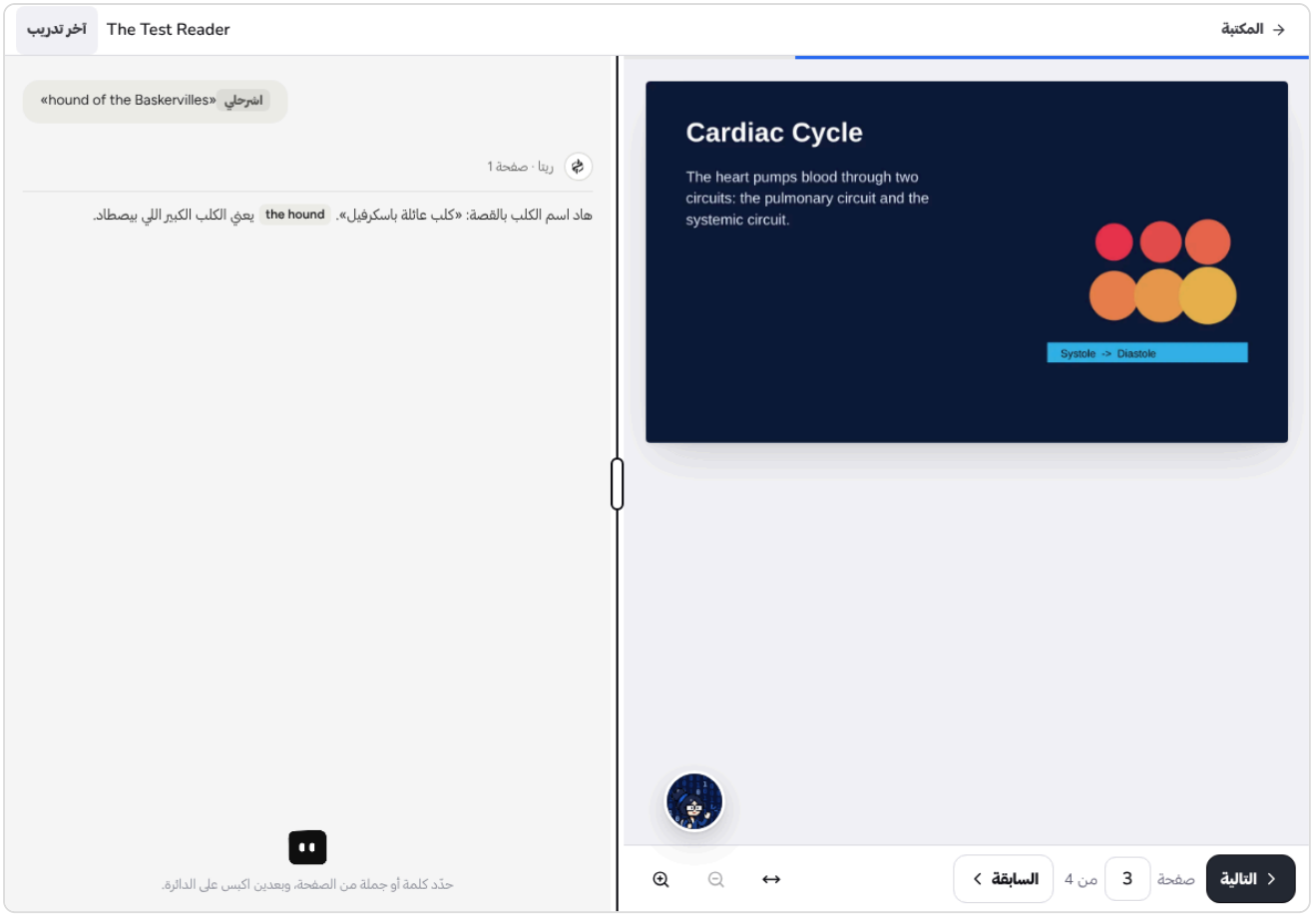
The invisible text layer made visible (red) to show it sits exactly over the picture.



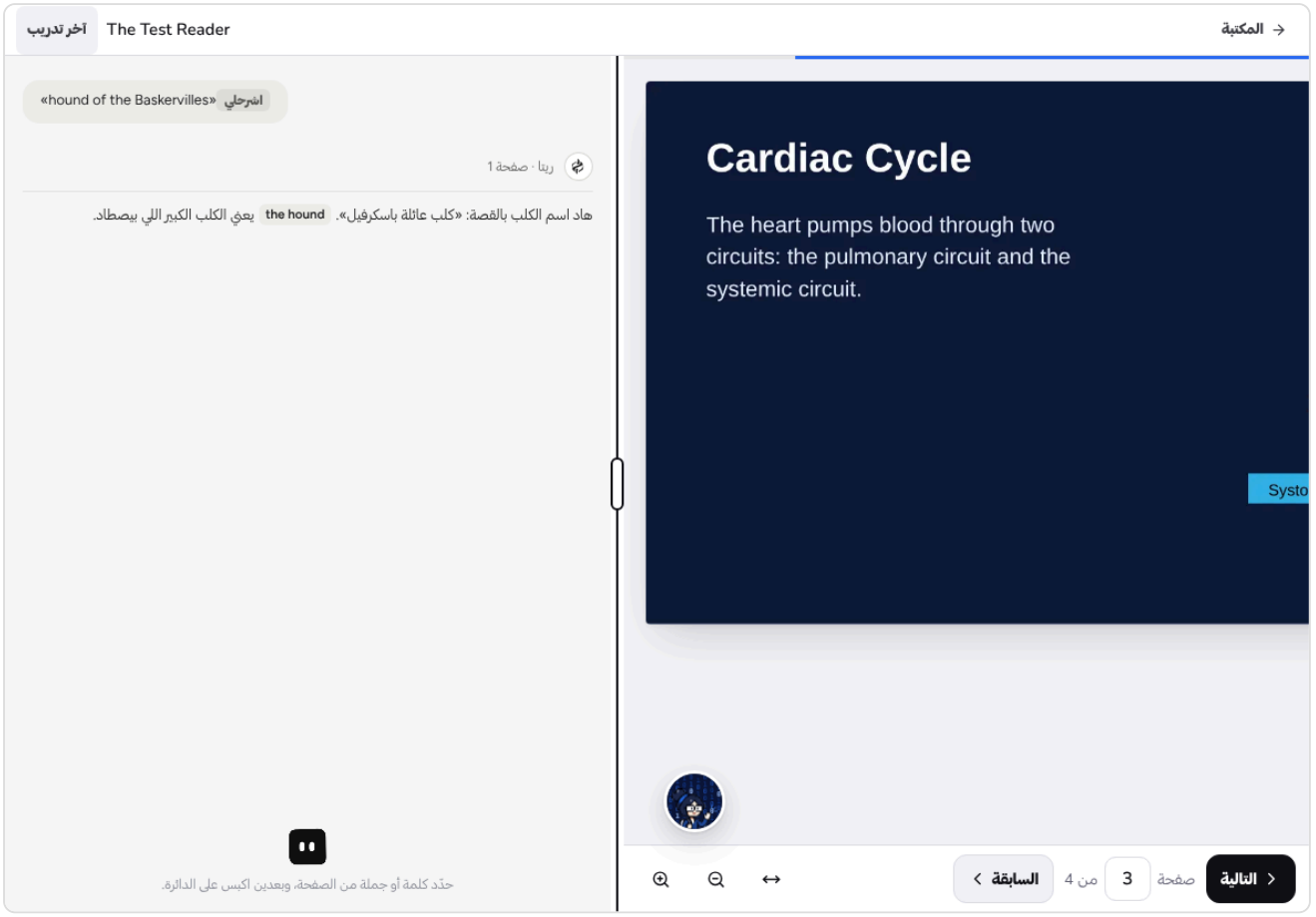
Selecting "hound of the Baskervilles" and opening the circle menu.



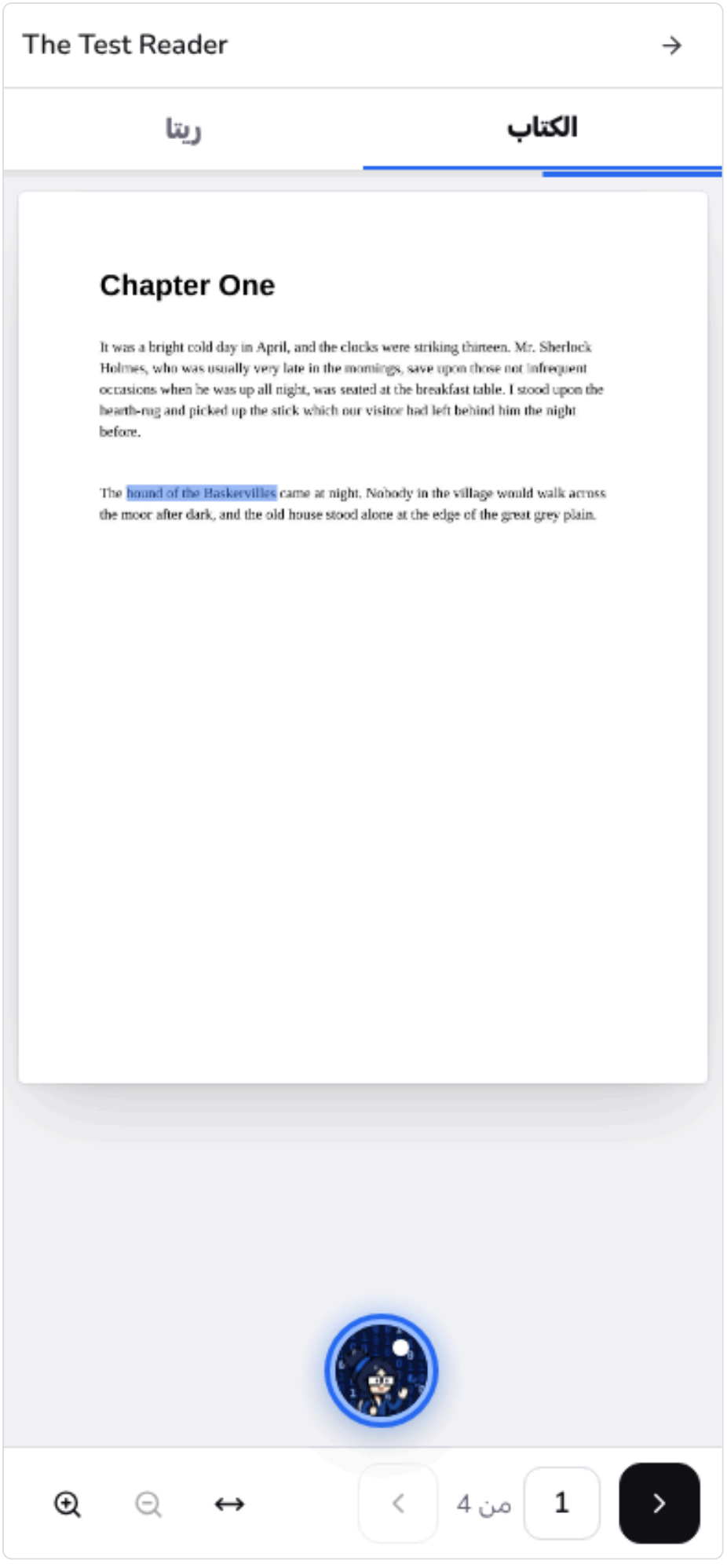
The tutor answers in writing in the left column.



A landscape lecture slide (pictures + text).



Zoomed 1.5×: the page scrolls inside the pane.



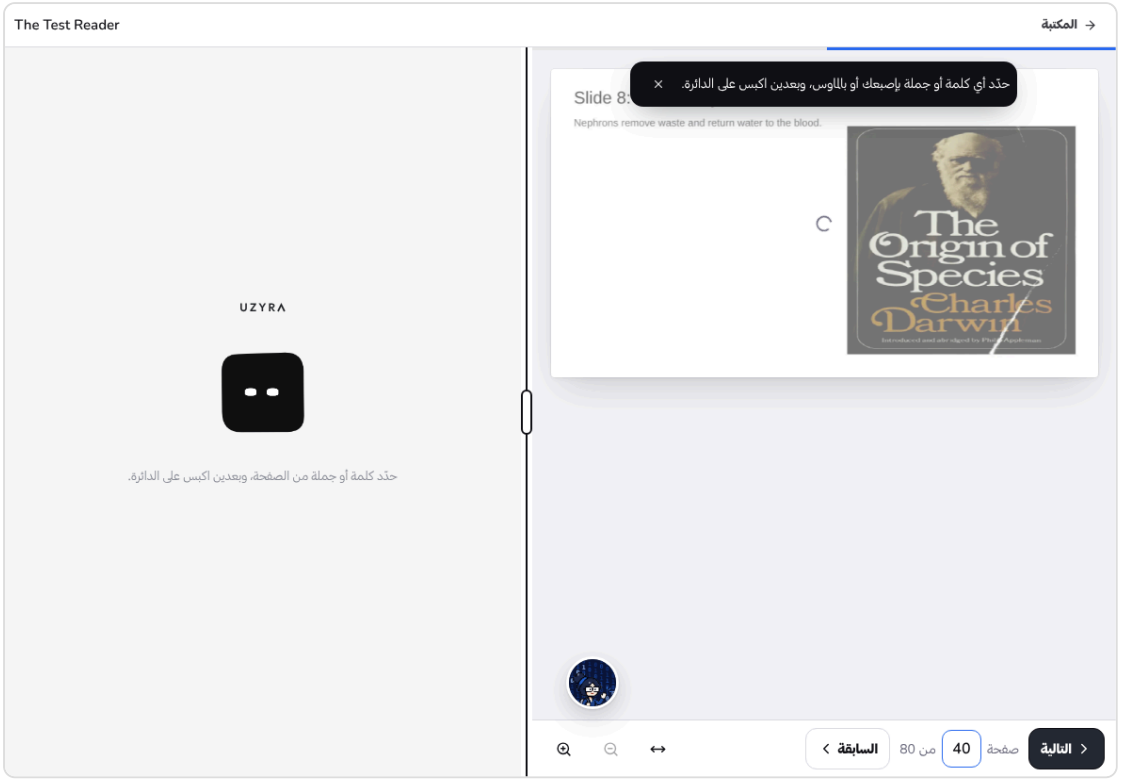
Phone (390×844): tabs «الكتاب» (book) / «ريتا» (tutor); the page is small, text needs zoom.



Phone: a picture-only (scanned) page.

2.3 Reading: when pages are loaded (the part we want to improve)

Moment	What happens now
Open the book	Read the book row (title, page_count), then the reader's saved page (or `?page=` in the URL). Load that page: ① row from `book_pages` (includes the full text + runs) → ② ask Storage for a 1-hour signed URL → ③ download the picture. These three are sequential.
After a page is shown	Prefetch **only page N+1 and N−1** (row + signed URL + picture warmed in the browser cache).
Next / previous	If prefetched: shown in 70–240 ms. The old page stays on screen (dimmed, small spinner) until the new picture has loaded, then the new one fades in (180 ms). Then N+1/N−1 of the new page are prefetched.
Fast flipping	Only one page ahead is ready; tapping «next» 5 times quickly (3→8) shows the final page 1.4 s after the first tap, with 16 requests.
Jump (type a page number)	Cold: 3 sequential requests ≈ 0.8 s with 250 ms latency (see the screenshot below: the old page stays dimmed with a spinner).
Cache	React Query keeps pages 45 min in memory. Each visit makes **new signed URLs**, so the browser's HTTP cache of pictures is not reused across visits/tabs.
Saving the place	After 1.2 s on a page: upsert into `book_reading_progress` and update `?page=`.



Jumping from page 8 to page 40 (not prefetched): the old page stays, dimmed, with a spinner, until page 40 arrives (≈0.8 s at 250 ms/request).

2.4 How the learner moves between pages today

- Buttons «التالية» / «السابقة» (44 px), arrow keys and Page Up/Down, typing a page number.
- **Not available yet:** swipe to turn, continuous vertical scrolling, two-page spread, thumbnails strip, table of contents / chapters, search inside the book, bookmarks, a scrubber (drag to page), pinch-zoom tuned for the page.

3. The problems we want solved

1. **How many pages should be shown at once?** One page (paged, like now), a continuous vertical scroll of pages (like Google Drive/pdf.js viewer/Scribd), or two pages side by side on landscape tablets? Different for novels vs lecture slides?
2. **How many pages to load and when?** Prefetch only ±1 now. What window (±2, ±3, adaptive to reading speed, direction-aware)? Batch the rows and signed URLs (Supabase supports `createSignedUrls` for many paths in one call, and one query can fetch several rows)? Smaller thumbnails first, then the sharp picture (progressive)? Separate the heavy text/runs from the picture so the picture can show first?
3. **Will loading page by page make the learner bored?** How do Kindle, Apple Books, Google Play Books, Perlego, Scribd, Everand, Readwise Reader, PSPDFKit/Nutrient, PDF.js handle instant page turns, fast flipping and jumps on mobile with limited data?
4. **How should the learner move around?** Swipe vs tap zones vs buttons (careful: swipes and taps must not conflict with selecting text), a page scrubber, thumbnails, chapters/TOC (we could extract the PDF outline when preparing), search, "continue where you left off".
5. **Phones:** a full PDF page is tiny on a 390 px screen. Should phones get a reflowed text view (we already store each page's text), fit-width with vertical scroll, or something else — while keeping selection for the tutor?
6. **Caching across visits**: stable URLs (e.g. a CDN with a long-lived token per book, or public bucket with unguessable paths?) vs privacy, within Supabase.

4. Numbers that matter

Item	Value
Page picture	1400 px wide WebP (JPEG on Safari), 8–86 KB measured, typical 50–100 KB
Text + runs per page	a few KB to tens of KB (up to 4,000 pieces / 60,000 chars per page)
Requests per new page	3 sequential (row, sign, picture) + 1 progress save
Prefetch window	±1 page
Book size	up to 5,000 pages / 500 MB PDF

5. Files (the full code is in the attached `uzyra-reader-code.md`)

File	Role
src/components/books/reader/BookReader.tsx	The reader page: layout, which page is shown, prefetch (±1), turning, zoom/fit, selection, the circle menu, the tutor's answers
src/components/books/reader/BookPageView.tsx	One page: the picture, the see-through text layer and its measuring, loading/failed states
src/components/books/reader/book-reader.css	The page, bar, progress line, text layer styles
src/lib/book-reader-data.ts	Loading a book, one page (row + signed URL), the reader's saved place
src/lib/book-prepare.ts	Admin preparation: render each page, upload, write rows, resume
src/lib/books-reader.ts	Pure helpers: text pieces from pdf.js, checking a selection against the page text

src/routes/books_.$bookId.read.tsx	The route /books/<book>/read
supabase/migrations/20261113120000_books_reader.sql	Tables book_pages, book_reading_progress, bucket book-pages and its access rules

6. Question to research

Please propose the best practical design for this web book reader (React, Supabase, iPad Safari and phones), respecting the constraints in section 1:

1. Paged vs continuous scroll vs two-page spread — per device and per kind of book (novel vs slides). Cite how leading readers do it.
2. A loading strategy so turning feels instant (target <100 ms for next/previous, <500 ms for a jump) and fast flipping never shows a blank page: prefetch window, batching rows and signed URLs, image sizes/thumbnails, priorities, cancelling unneeded requests, memory limits on iPad.
3. Navigation that does not conflict with text selection: swipe/tap zones, scrubber, thumbnails, outline/TOC, search, resume.
4. The phone experience (small screen) while keeping selectable text for the tutor.
5. Caching pictures across visits with private files and signed URLs in Supabase Storage.

Give concrete numbers, libraries if useful (e.g. virtualization, gesture libraries), and the trade-offs.